

□□□□ >

Doom [rspress](#) [Alauda](#)

□ □ □ □

- □□□□□□□□□□□□□□□□
- □□□□□□□
- □□□□□

MDX `<code></code>` **Markdown** `<code></code>` **Markdown** `<code></code>`

`<code></code>` **React** `<code></code>`



0000000000 MDX 00 ↗

0000

0000000000 Doom 00

00

00

0000

000000

00

□□ `doom` □□□□

□ □ □ □

□ □ □ □ □ □ □ □

□ □ □ □ □ □

Page 10 of 10

--	--	--	--	--	--	--	--

□ □ □ □

Page 10 of 10

Sitemap ☐ ☐

[illegible]

3/3

11

11

Markdown

Callouts

Mermaid

📄📄📄

MDX

📄 MDX 📄📄📄📄📄📄📄📄📄📄

rspress 📄

doom 📄

📄📄📄📄📄

📄📄

📄📄📄📄📄📄📄📄📄

i18n.json

.ts/.tsx

.mdx

API 📄

📄 API

CRD

📄📄📄

📄 openapi 📄

📄📄📄📄📄

props

📄

0000

000000

00

00000000000000000000 ACP 00

00000

00000

000000

00000000

📄📄

📄📄

📄📄📄

📄📄📄📄

📄📄📄📄📄

📄📄📄📄📄

📄📄📄

📄📄📄📄📄📄

📄📄📄

📄📄 PDF

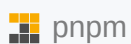
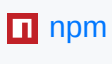
📄📄📄

📄📄📄📄

📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

```
mkdir my-docs && cd my-docs
```

📄 `npm init -y` 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄 `npm`📄`yarn` 📄 `pnpm` 📄📄 `doom:`



```
npm install -D @alauda/doom typescript
```

📄📄📄📄📄📄📄📄:

```
# 创建 docs 目录并初始化
mkdir docs/en && echo '# Hello World' > docs/en/index.md
mkdir docs/zh && echo '# 文档' > docs/zh/index.md
```

在 `package.json` 中添加脚本:

```
{
  "scripts": {
    "dev": "doom dev",
    "build": "doom build",
    "new": "doom new",
    "serve": "doom serve",
    "translate": "doom translate",
    "export": "doom export"
  }
}
```

在 `doom.config.yml` 中添加:

```
title: My Docs
```

在 `tsconfig.json` 中添加:

```
{
  "compilerOptions": {
    "jsx": "react-jsx",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "strict": true,
    "target": "ESNext",
  },
  "mdx": {
    "checkMdx": true,
  },
}
```

global.d.ts

```
/// <reference types="@alauda/doom/runtime" />
```

.mdx doom


```
doom -h
```

```
# output
```

```
Usage: doom [options] [command]
```

```
Doctor Doom making docs.
```

Options:

<code>-V, --version</code>	output the version number
<code>-c, --config <config></code>	Specify the path to the config file
<code>-v <version></code>	Specify the version of the documentation, c
<code>-b, --base <base></code>	Override the base of the documentation
<code>-p, --prefix <prefix></code>	Specify the prefix of the documentation bas
<code>-f, --force [boolean]</code>	Force to 1. fetch latest reference remotes or scaffo 2. translate ignore hash equality check and
<code>-i, --ignore [boolean]</code>	Ignore internal routes (default: false)
<code>-d, --download [boolean]</code>	Display download pdf link on nav bar (defau
<code>-e, --export [boolean]</code>	Run or build in exporting PDF mode, `apis/*
<code>-I, --include <language...></code>	Include only the specific language(s),
<code>-E, --exclude <language...></code>	Include all languages except the specific l
<code>-o, --out-dir <path></code>	Override the `outDir` defined in the config
<code>-r, --redirect <enum></code>	Whether to redirect to the locale closest t
<code>-R, --edit-repo [boolean url]</code>	Whether to enable or override the `editRepo
<code>-a, --algolia [boolean alauda]</code>	Whether to enable or use the alauda (docs.a
<code>-S, --site-url</code>	Whether to enable the siteUrl for sitemap g
<code>-n, --no-open</code>	Do not open the browser after starting the
<code>-h, --help</code>	display help for command

Commands:

<code>dev [options] [root]</code>	Start the development server
<code>build [root]</code>	Build the documentation
<code>preview serve [options] [root]</code>	Preview the built documentation
<code>new [template]</code>	Generate scaffolding from templates
<code>translate [options] [root]</code>	Translate the documentation
<code>export [options] [root]</code>	Export the documentation as PDF, `apis/**`
<code>lint [root]</code>	Lint the documentation
<code>help [command]</code>	display help for command

📄📄📄📄📄

📄 `yarn dev` 📄📄📄📄📄📄📄📄📄📄📄📄📄📄

```
doom dev -h
```

```
# output
```

```
Usage: doom dev [options] [root]
```

```
Start the development server
```

```
Arguments:
```

```
  root                                Root directory of the documentation
```

```
Options:
```

```
  -H, --host [host]                  Dev server host name
  -P, --port [port]                  Dev server port number
  -l, --lazy [boolean]               Whether to enable `lazyCompilation` which could i
  --no-lazy                          Do not enable `lazyCompilation`
  -h, --help                         display help for command
```

📄📄📄📄📄

📄 `yarn build` 📄📄📄📄📄📄📄📄📄 `dist` 📄📄📄📄📄

📄📄📄

📄 `yarn serve` 📄📄📄📄📄📄📄📄📄📄 `-b` , `-p` 📄📄📄📄📄📄📄📄 `-b` , `-p` 📄
📄

📄📄📄📄📄📄

📄 `yarn new` 📄📄📄📄📄📄📄📄📄📄

📄📄📄

```
doom translate -h
```

```
# output
```

```
Usage: doom translate [options] [root]
```

Translate the documentation

Arguments:

root Root directory of the documentation

Options:

-s, --source <language> Document source language, one of en, zh, ru (default is en)

-t, --target <language> Document target language, one of en, zh, ru (default is en)

-g, --glob <path...> Glob patterns of source dirs/files to translate

-C, --copy [boolean] Whether to copy relative assets to the target directory

-h, --help display help for command

- -g, --glob 指定要翻译的源目录/文件的glob模式 glob 模式支持通配符

1. yarn translate -g abc xyz 将 <root>/<source>/abc , <root>/<source>/xyz 复制到 <root>/<target>/abc , <root>/<target>/xyz

2. yarn translate -g '*' 将 <root>/<source> 下的所有文件复制到 <root>/<target>

- -C, --copy 是否复制源目录/文件中的相对资产到目标目录 false 表示不复制

- 相对资产

1. /<source>/abc.jpg 复制到 <root>/public/<source>/abc.jpg 和 <root>/public/<target>/abc.jpg 复制到 <root>/<target>/abc.jpg

2. <root>/<source>/abc.mdx 复制到 ./assets/xyz.jpg 复制到 <root>/<source>/assets/xyz.jpg 复制到 <root>/<target>/assets/xyz.jpg 复制到 <root>/<target>/assets/xyz.jpg

3. <root>/<source>/abc.mdx 复制到 ./assets/<source>/xyz.jpg 复制到 <root>/<source>/assets/<source>/xyz.jpg 复制到 <root>/<target>/assets/<target>/xyz.jpg 复制到 <root>/<target>/assets/<target>/xyz.jpg

- 📁📁📁📁📁📁

1. `/<source>/abc.jpg` 📁📁📁📁 `<root>/public/<target>/abc.jpg` 📁📁📁📁📁
📁📁📁📁📁📁 `/<target>/abc.jpg` 📁📁📁📁📁📁📁📁📁📁📁📁
2. `<root>/<source>/abc.mdx` 📁📁📁 `./assets/<source>/xyz.jpg` 📁📁📁📁📁
`<root>/<target>/assets/<target>/xyz.jpg` 📁📁📁📁📁📁📁📁📁📁📁📁
`./assets/<target>/xyz.jpg` 📁📁📁📁📁
`../<source>/assets/<target>/xyz.jpg`

WARNING

📁📁📁📁📁📁 `-g '*'` 📁📁📁📁📁📁📁📁 `source` 📁 `target` 📁📁📁📁📁📁
`internalRoutes` 📁📁📁📁📁📁 `target` 📁📁📁📁📁📁

TIP

📁📁📁📁📁📁📁 `AZURE_OPENAI_API_KEY` 📁📁📁📁📁📁📁📁📁📁📁 `Leader` 📁📁

📁📁📁📁📁📁📁📁📁📁📁📁📁📁

`i18n:`
 `title:`
 `en: DevOps Connectors`
 `additionalPrompts: '📁📁📁 Connectors 📁📁📁📁📁📁'`
 `disableAutoTranslation: false`
`title: DevOps 📁📁`

📁📁📁📁📁📁📁📁📁

📁📁 **PDF**

WARNING

📁📁📁📁📁📁📁📁📁 `yarn build` 📁📁📁

```
doom export -h
```

```
# output
```

```
Usage: doom export [options] [root]
```

```
Export the documentation as PDF, `apis/**` and `*/apis/**` routes will be ign
```

```
Arguments:
```

```
  root                Root directory of the documentation
```

```
Options:
```

```
  -H, --host [host]  Serve host name
```

```
  -P, --port [port]  Serve port number (default: "4173")
```

```
  -h, --help         display help for command
```

```
❯ yarn export 📄📄📄 PDF 📄📄📄📄📄📄📄 -b , -p 📄📄📄📄📄📄📄📄 -b , -p
❯
```

```
📄📄📄📄 playwright ↗📄📄📄📄📄 build-harbor.alauda.cn/frontend/playwright-
runner:doom 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄
```

```
.env.yarn
```

```
PLAYWRIGHT_DOWNLOAD_HOST="https://cdn.npmmirror.com/binaries/playwright"
```

```
📄📄📄📄📄📄📄📄 pdf 📄📄📄 doom 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄
```

```
📄📄📄📄
```

```
doom lint -h
```

```
# output
```

```
Usage: doom lint [options] [root]
```

```
Lint the documentation
```

```
Arguments:
```

```
  root                Root directory of the documentation
```

```
Options:
```

```
  -g, --glob <path...>  Glob patterns of source dirs/files to lint (default:
  --no-cspell            Disable cspell linting
  -h, --help            display help for command
```

```
doom lint ESLint cspell
```

[ESLint](#) / [CSpell](#):

- `eslint.config.mjs`:

```
import doom from '@alauda/doom/eslint'

export default doom(new URL('docs', import.meta.url))
```

- `cspell.config.mjs`:

```
export { default } from '@alauda/doom/cspell'
```

`CWD` `.cspell` `CSpell`

`.cspell/k8s.txt`

```
k8s
kubernetes
```



📄

📄 `doom` 📄📄

📄📄

📄📄

API 📄📄

📄📄📄📄

📄📄📄

📄📄📄

📄📄📄

📄📄📄📄

📄📄📄📄📄

📄📄📄📄

`sites.yaml` 📄

📄📄

📄📄📄📄📄

📄📄📄

📄📄📄

Algolia 📄📄

Sitemap 📄

API 00

00 API

CRD

0000

00 openapi 00

000000

props

00

0000

000000

00

00000000000000000000 ACP 00

00000

00000

000000

00000000

📄📄

📄📄

🔗🔗🔗

🔗🔗🔗

API 🔗🔗🔗

🔗🔗🔗🔗🔗🔗

🔗🔗🔗🔗🔗

frontmatterMode

🔗🔗🔗🔗🔗

🔗🔗🔗🔗

🔗🔗🔗🔗🔗🔗

🔗🔗🔗🔗🔗🔗🔗

🔗🔗🔗🔗🔗🔗

sites.yaml 🔗🔗

🔗🔗🔗🔗

🔗🔗🔗🔗🔗🔗🔗

🔗🔗🔗🔗🔗

🔗🔗🔗🔗🔗

Algolia 🔗🔗🔗🔗

Sitemap 🔗🔗

📄📄📄📄

```

doom.config.yaml doom.config.yaml
doom.config.yaml rspress js/ts
.js/.ts/.mjs/.mts/.cjs/.cts
js/ts @alauda/doom/config defineConfig

```

```
import { defineConfig } from '@alauda/doom/config'

export default defineConfig({})
```

□ □ □ □

- `lang` `en` `null` `undefined`
- `title`
- `logo` `public` `doom` `alauda logo`
- `logoText` `logo`
- `icon` `favicon` `logo`
- `base` `product-docs` `/`
- `outDir` `dist/{base}/{version}` `dist/{outDir}/{version}` `version`

API ☐ ☐ ☐ ☐

api:

```
# CRD 文档目录 doom.config.* 文档目录 glob docs/json/yaml 文档
```

crds:

```
- docs/shared/crds/*.yaml
```

```
# OpenAPI 文档目录 doom.config.* 文档目录 glob docs/json/yaml 文档
```

openapis:

```
- docs/shared/openapis/*.json
```

```
# 文档 openapi 文档目录 docs/shared/openapis/*.json
```

```
# 文档 https://doom.alauda.cn/apis/references/CodeQuality.html#v1alpha1.CodeQ
```

references:

```
v1alpha1.CodeQualityBranch: /apis/references/CodeQualityBranch#v1alpha1.C
```

```
# 文档API 文档目录 docs/shared/gateway 文档目录 docs/shared/gateway
```

```
pathPrefix: /apis
```

文档目录 API 文档

文档目录

```
# 文档目录 doom.config.* 文档目录 glob docs/json/yaml 文档
```

permission:**functionresources:**

```
# `kubectl get functionresources`
```

```
- docs/shared/functionresources/*.yaml
```

roletemplates:

```
# `kubectl get roletemplates -l auth.cpaas.io/roletemplate.official=true`
```

```
- docs/shared/roletemplates/*.yaml
```

文档目录

文档目录

reference:

```
- repo: alauda-public/product-doc-guide # 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄
branch: # [string] 📄📄📄📄📄📄📄
publicBase: # [string] 📄📄📄📄📄📄📄📄 /images/xx.png 📄📄📄📄📄📄📄📄 do
sources:
  - name: anchor # 📄📄📄📄📄📄📄📄📄📄📄
    path: docs/index.mdx#📄📄 # 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄 doom.conf
    ignoreHeading: # [boolean] 📄📄📄📄📄📄📄 true📄📄📄📄📄📄📄📄📄📄📄
    processors: # 📄📄📄📄📄📄📄
      - type: ejsTemplate
        data: # ejs 📄📄📄📄📄 `<%= data.xx %>` 📄📄
    frontmatterMode: merge # 📄📄📄📄📄📄 frontmatter 📄📄📄📄📄 ignore📄📄📄📄📄 ign
```

frontmatterMode

- ignore 📄📄📄📄📄📄 frontmatter📄📄📄📄📄📄📄 frontmatter
- merge 📄📄📄📄📄📄 frontmatter📄📄📄📄📄 key📄📄📄📄📄📄📄📄📄📄📄📄📄
- replace 📄📄📄📄📄📄 frontmatter 📄📄📄📄📄 frontmatter
- remove 📄📄📄📄📄📄 frontmatter

📄📄📄📄📄📄📄📄

📄📄📄📄📄📄

releaseNotes:

```
queryTemplates:
  fixed: # 📄📄 ejs 📄📄 jql 📄📄
  unfixed:
```

release-notes.md

<!-- release-notes-for-bugs?template=fixed&project=Dev0ps -->

release-notes.mdx

```
{/* release-notes-for-bugs?template=fixed&project=DevOps */}
```

```

template=fixed&project=Dev0ps  fixed  queryTemplates 
query  project=Dev0ps  ejss  fixed  jira  jql 
https://jira.alauda.cn/rest/api/2/search?jql=<jql>  API 
JIRA_USERNAME  JIRA_PASSWORD 

```



```
sidebar:
  collapsed: false # [] false
```



```
internalRoutes: # 忽略 glob 匹配的 docs 以及 cli 的 '-i, --ignore' 选项
  - '*/internal/**'
```

--	--	--	--	--	--	--	--	--

```
onlyIncludeRoutes: # 只包含 glob 匹配的 docs 中的 cli 子目录，忽略其他子目录
  - '*/internal/**'

internalRoutes:
  - '*/internal/overview.mdx'
```

```
theme: # optional, https://shiki.style/themes
langs: # optional, https://shiki.style/languages
transformers: # optional, only available in js/ts config, https://shiki.style
```

plaintext

```
sites.yaml [REDACTED]
```

--	--	--	--

translate:

```
# 将sourceLang `sourceLang`, `targetLang`, `userPrompt`, `additional
# 将 `sourceLang` 与 `targetLang` 与 `源` 与 `目标` 语言
# `userPrompt` 语言
# `additionalPrompts` 与 `frontmatter.il8n` 与 `additionalPrompts` 语言
# `terms` 与 `titleTranslationPrompt` 语言 AI 语言
# 语言
```

systemPrompt: |

You are a professional technical documentation engineer, skilled in writing h

Baseline Requirements

- Sentences should be fluent and conform to the expression habits of the <%=
- Input format is MDX; output format must also retain the original MDX format
- ****CRITICAL****: Do not translate or modify ANY link content in the document.
 - URLs in markdown links: [text](URL) - keep URL exactly as is
 - Reference-style links: [text][ref] and [ref]: URL - keep both ref and URL
 - Inline URLs: https://example.com - keep completely unchanged
 - Image links: ![alt](src) - keep src unchanged, but alt text can be transl
 - Anchor links: [text](#anchor) - keep #anchor unchanged
 - Any href attributes in HTML tags - keep unchanged
- Do not translate professional technical terms and proper nouns, including b
- The title field and description field in frontmatter should be translated,
- Content within MDX components needs to be translated, whereas MDX component
- Do not modify or translate any placeholders in the format of __ANCHOR_N__ (
- Keep original escape characters like backslash, angle brackets, etc. unchan
- Do not add any escape characters to special characters like [], (), {}, etc
 - If source has "Architecture [Optional]", keep it as "Architecture [Option
 - If source has "Function (param)", keep it as "Function (param)" (not "Fun
 - Only add escape characters if they were present in the original text
- Preserve and do not translate the following comments, nor modify their cont
 - { /* release-notes-for-bugs */ }
 - <!-- release-notes-for-bugs -->
- Remove and do not retain the following comments:
 - { /* reference-start */ }
 - { /* reference-end */ }
 - <!-- reference-start -->
 - <!-- reference-end -->
- Ensure the original Markdown format remains intact during translation, such
- Do not translate the content of the code block.

<% if (titleTranslationPrompt) { %>

<%- titleTranslationPrompt %>

<% } %>

<% if (terms) { %>


```

<%- terms %>
<% } %>

<% if (userPrompt || additionalPrompts) { %>
## Additional Requirements
These are additional requirements for the translation. They should be met also

The text for translation is provided below, within triple quotes:
"""
<% if (userPrompt) { %>
<%- userPrompt %>
<% } %>

<% if (additionalPrompts) { %>
<%- additionalPrompts %>
<% } %>
"""
<% } %>

```

🔒🔒🔒🔒🔒🔒🔒🔒

editRepoBaseUrl: [alauda/doom/tree/main/docs](https://github.com/alauda/doom/tree/main/docs) # [https://github.com/](https://github.com/alauda/doom/tree/main/docs) 🔒🔒🔒🔒🔒🔒🔒🔒

🔒🔒🔒🔒🔒🔒

```

export:
- name: Concepts # 🔒🔒🔒🔒 pdf 🔒🔒🔒🔒🔒🔒
  scope: '*/concepts' # 🔒🔒🔒🔒🔒🔒🔒🔒🔒🔒 glob 🔒🔒🔒🔒 docs 🔒

```

🔒🔒🔒🔒🔒🔒

```
lint:
  cspellOptions: # cspell cspell https://github.com/streetsidesoftware/csp
```

Algolia ☐☐☐☐

```
algolia: # Algolia 配置参数 -a, --algolia` 配置参数
  appId: # Algolia ID
  apiKey: # Algolia API Key
  indexName: # Algolia 索引名
```

public/robots.txt Algolia

INFO

rspress Algolia [🔗](#)
@alauda/doom/theme

theme/index.ts

```
export * from '@alauda/doom/theme'
```

Sitemap

```
siteUrl: https://docs.alauda.cn # 指定 URL 生成 sitemap -S, --site-url
```

11

11

□ □ □ □

111

11

11

index

index.md index.mdx

```
├─ index.md
├─ start.mdx
└─ usage
    ├─ index.mdx
    └─ convention.md
```

□ □ □ □ □ □

- [illegible]

🐱🐱

🐱🐱🐱🐱🐱🐱 `frontmatter` 🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱

```
---
title: 🐱
description: 🐱
author: 🐱
category: 🐱
---
```

🐱🐱🐱🐱🐱🐱 `MDX` 🐱 `.mdx` 🐱🐱🐱🐱🐱 `frontmatter` 🐱🐱🐱🐱🐱🐱

🐱🐱

🐱 `index.md` 🐱 `index.mdx` 🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱 `frontmatter` 🐱
`weight` 🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱 `weight` 🐱🐱🐱🐱🐱🐱

```
---
weight: 1
---
```

WARNING
🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱🐱

🐱🐱

index.mdx Overview

```
#  
  
<Overview />
```

1. `;;, %, #, //, /** */, --` `<!-- -->`
2. `[\\!code callout]`
3. `:::callouts` `<div class="doom-callouts">` `<Callouts>`

```
```sh
```

```
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
 + 218 MiB \ # [\\!code callout]
 + 8 MiB × (number of vCPUs) \ # [\\!code callout]
 + 16 MiB × (number of graphics devices) \ # [\\!code callout]
 + (additional memory overhead) # [\\!code callout]
```

```
```
```

```
:::callouts
```

1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
 - If your environment includes a Single Root I/O Virtualization (SR-IOV) n
 - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
 - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

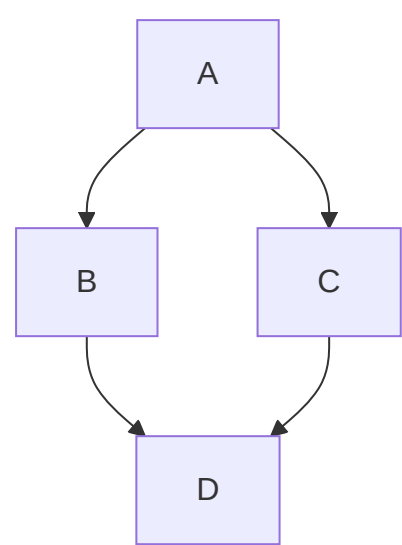
```
:::
```

```
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
    + 218 MiB \ ①
    + 8 MiB × (number of vCPUs) \ ②
    + 16 MiB × (number of graphics devices) \ ③
    + (additional memory overhead) ④
```

- ① Required for the processes that run in the `virt-launcher` pod.
- ② Number of virtual CPUs requested by the virtual machine.
- ③ Number of virtual graphics cards requested by the virtual machine.
- ④ Additional memory overhead:
 - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
 - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
 - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

Mermaid ↗

```
```mermaid
graph TD;
 A-->B;
 A-->C;
 B-->D;
 C-->D;
```
```



[Markdown Preview Mermaid](#) VSCode

WARNING

.md [issue](#)

```
{width="100" height="100"}
```




MDX

[MDX](#) [Markdown](#) [Markdown](#) [JSX](#) [rspress MDX](#)

rspress

doom

Overview

Directive

ExternalSite

ExternalSiteLink

AcpApis0verview ExternalApis0verview

Term

props

TermsTable

props

JsonViewer

rspress

rspress [rspress](#) [.mdx](#)

- Badge
- Card

- [LinkCard](#)
- [PackageManagerTabs](#)
- [Steps](#)
- [Tab/Tabs](#)
- [Toc](#)

在 `rspress/theme` 中配置

```
preview.mdx

import { SourceCode } from '@rspress/core/theme'

<SourceCode href="/" />
```

doom 文档

`doom` 是一个用于生成文档的工具

Overview

文档生成工具

Directive

在文档中使用 `Directive` 来定义

```
- 在 `doc/en` 中定义 `doc/zh` 的文档内容

<Directive type="danger" title="危险">
  在 `doc/zh` 中定义文档内容
</Directive>
```

- `doc/en` `doc/zh`

11

doc/zh

11

ExternalSite

--	--	--	--	--	--	--	--

```
<ExternalSite name="connectors" />
```

Note

























DevOps ☐ ☐ ☐ ☒

ExternalSiteLink

□ □ □ □ □ □ □ □ □ □

```
<ExternalSiteLink name="connectors" href="link.mdx#hash" children="Content" /
```

[Content ↗](#)

TIP

```
mdx <ExternalSiteLink name="connectors" href="link" children="Content" />
```

□ □ □ □ □ □ □ □ □ □

```
<ExternalSiteLink name="connectors" href="link">
  Content {/* `p` */}
</ExternalSiteLink>
```

p children

AcpApisOverview ExternalApisOverview

API

```
<AcpApisOverview />
{/* same as following */}
<ExternalApisOverview name="acp" />

<ExternalApisOverview name="connectors" />
```

Note

ACP APIs [ACP APIs](#)

Note

DevOps APIs [DevOps APIs](#)

Term

```
<Term name="company" textCase="capitalize" />
<Term name="product" textCase="lower" />
<Term name="productShort" textCase="upper" />
```

ACP

props

- `name` : `□□□□□□□□□□□□□□□□`
- `textCase` : `□□□□□□□□□□□□` `lower` , `upper` , `capitalize`

TermsTable



```
<TermsTable />
```

公司	名称	类型	名称	类型	名称
company	名称	-	Alauda	-	名称
product	名称 名称	-	Alauda Container Platform	-	名称
productShort	ACP	-	ACP	-	名称 名称

props

- terms : NormalizedTermItem[] □□□□□□□□□□□□□□□□□□□□□□□□

JsonViewer

```
<JsonViewer value={{ key: 'value' }} />
```

```
yaml      json
key:    value
```

□□□□□□□□

□□□□□□□□□□□□□□□□□□ `shared` □□□□□□□□□□□□□□□□□□

```
import CommonContent from './shared/CommonContent.mdx'

<CommonContent />
```

□□□□□□□□ `runtime` [↗](#) □□□ API□□□□□□ `.jsx/.tsx` □□□□□□□□ `.mdx` □□□□□□□□

```
// shared/CommonContent.tsx
export const CommonContent = () => {
  const { page } = usePageData()
  return <div>{page.title}</div>
}

// showcase/content.mdx
import { CommonContent } from './shared/CommonContent'
<CommonContent />
```

WARNING

□□□□□ `.mdx` □□□□□□□□□ `props` □□□□□□ [issue](#) [↗](#) □□□□□□□□ `props` □□□□□□□□
`.jsx/.tsx` □□□□□□□□

📄📄📄

`alauda` 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄 `en / zh` 📄📄📄📄📄📄📄📄📄📄📄📄
📄 `public` 📄📄📄 `en / zh` 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

📄📄

- `i18n.json`
- `.ts/.tsx`
- `.mdx`

`i18n.json`

📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄 `docs` 📄📄📄 `i18n.json` 📄📄
📄📄📄📄📄 `useI18n` 📄📄📄📄📄📄📄📄

docs/i18n.json

```
{
  "title": {
    "zh": "标题",
    "en": "Title"
  },
  "description": {
    "zh": "描述",
    "en": "description"
  }
}
```


.ts/.tsx

```
import { useI18n } from '@rspress/core/runtime'

export const CommonContent = () => {
  const t = useI18n()
  return <h1>{t('title')}</h1>
}
```

.mdx

```
import { useI18n } from '@rspress/core/runtime'

# {useI18n()}('title')}

{useI18n()}('description')}
```

□□□□□□□□□□ **API** □□□□ **API** □ **CRD (Custom Resource Definition)** □□□□□□□□□□
□□□□□□

11

□□ openapi □□

```
<OpenAPIPath path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summ
```

CodeQualityTaskSummary

props

- path : OpenAPI schema paths
- pathPrefix : api.pathPrefix
- openapiPath : openapi

CRD

```
crds/ArtifactCleanupRun.mdx

# ArtifactCleanupRun

<K8sCrd name="artifactcleanupruns.artifacts.katanomi.dev" />
```

ArtifactCleanupRun

props

- name : CRD metadata.name
- crdPath : openapi CRD

```
references/CodeQuality.mdx

# CodeQuality

<OpenAPIRef schema="v1alpha1.CodeQuality" />
```

CodeQuality

props

- `schema` : OpenAPI schema `definitions` (v2) or `component/schemas` (v3) のスキーマ
- `openapiPath` : `openapi` のパス

openapi

`OpenAPIPath` と `OpenAPIRef` を指定して `openapi` を参照する。
`openapi` のパスを `openapiPath` で指定する。

```
<OpenAPIPath  
  path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary"  
  openapiPath="shared/openapis/katanomi.json"  
>
```

权限管理

```

<K8sPermissionTable functions={['devops-testplans', 'devops-testmodules']} />

```

权限

props

权限

props

- functions: string[] - 权限资源 FunctionResource 权限资源

权限

权限	权限	权限 权限	权限 权限	权限 权限	权限 权限	权限 权限	权限 权限
权限 devops-testplans	权限	✓	✓	✓	✓	✓	×
	权限	✓	×	✓	✓	✓	×
	权限	✓	×	✓	✓	✓	×
	权限	✓	×	✓	✓	✓	×

测试用例	测试用例	测试用例	测试用例	测试用例	测试用例	测试用例	测试用例
测试用例 devops- testmodules	测试用例	测试用例 测试用例	测试用例 测试用例	测试用例 测试用例	测试用例 测试用例	测试用例 测试用例	测试用例 测试用例
	测试用例	✓	✓	✓	✓	✓	✗
	测试用例	✓	✗	✓	✓	✓	✗
	测试用例	✓	✗	✓	✓	✓	✗
	测试用例	✓	✗	✓	✓	✓	✗

☐ Markdown ☐☐☐☐

```
<!-- reference-end -->
```

```
{/* reference-end */}
```

```
□□□□□ <!-- reference-start#ref --> □□□□□
```

frontmatterMode

--	--	--	--	--	--

reference:

```

- repo: alauda-public/product-doc-guide # 文档仓库地址
branch: # [string] 分支名称
publicBase: # [string] 文档仓库地址 /images/xx.png 文档仓库地址 do
sources:
- name: anchor # 文档名称
path: docs/index.mdx#[] # 文档地址 doom.conf
ignoreHeading: # [boolean] 是否忽略标题 true 忽略
processors: # 处理器
- type: ejsTemplate
data: # ejs 模板 `<%= data.xx %>` 模板
frontmatterMode: merge # 是否合并 frontmatter 是否忽略 ignore 忽略

```

frontmatterMode

- **ignore** 忽略 frontmatter
- **merge** 合并 frontmatter key 重复的 key 会合并
- **replace** 替换 frontmatter
- **remove** 移除 frontmatter

文档地址


```
doom build # 
doom serve # 
```

doom build dist -v

```
# 4.0 4.0
```

```
doom build -v 4.0 # 4.0 dist/4.0 {base}/4.0
```

```
doom build -v master # master dist/master {base}/master
```

```
doom build -v {other} # dist/{other} {base}/{other}
```

```
# unversioned unversioned-x.y
```

```
doom build -v unversioned # dist/unversioned {base}
```

```
doom build -v unversioned-4.0 # 4.0 dist/unversioned-4.0
```

□□□□□□

```
|— console-platform
|   |— 4.0
|   |— 4.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
|— console-devops-docs
|   |— 4.0
|   |— 4.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
|— console-tekton-docs
|   |— 1.0
|   |— 1.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
```

index.html

```
overrides.yaml

# 钉钉群机器人配置
title:
  en: Doom - Alauda
  zh: Doom - 钉钉
logoText:
  en: Doom - Alauda
  zh: Doom - 钉钉
```

```
versions.yaml
```

- '4.1'
- '4.0'